

ALMAYA

Plateforme touristique — Rapport d'application

Un audit complet du code source React / Express / MySQL : architecture, périmètre fonctionnel, modèle de données et failles de sécurité vérifiées.

Application développée par : KOURITA Zounoogo Oussama Mahomet

Client : ALMAYA — agence de services et d'expériences touristiques (Maroc)

Périmètre : frontend/ , backend/ , Database/ , configurations de déploiement, historique git

Méthode : lecture directe du code source et vérification croisée avec la documentation existante (corrigée si erronée)

Date : juillet 2026

Ce rapport reflète l'état du dépôt au moment de l'audit.

1. Présentation de l'application

ALMAYA est une plateforme touristique marocaine complète : un frontend React (application monopage) et un backend Express/MySQL, avec une authentification JWT basée sur des cookies. Les visiteurs s'inscrivent, vérifient leur adresse e-mail, se connectent, parcourent les catégories touristiques, les destinations et les offres, constituent un panier, puis finalisent leur commande en envoyant celle-ci sous forme de message WhatsApp pré-rempli. Il existe également une page de « paiement simulé » purement décorative — ce n'est pas une véritable intégration de paiement. Les administrateurs disposent d'écrans CRUD pour les catégories, les destinations et les offres.

2. Pile technologique

Couche	Technologie
Frontend	React 19, React Router 7, Bootstrap 5, jwt-decode (importé mais plus utilisé)
Backend	Node/Express 5, MySQL2, bcrypt, jsonwebtoken, Nodemailer, express-rate-limit, cookie-parser
Base de données	MySQL (hébergée sur Railway en production)
Hébergement	Backend → Railway (build Nixpacks) ; Frontend → Vercel
E-mail	SMTP Gmail via Nodemailer (liens de vérification + formulaire de contact)

3. Structure du dépôt

```
Almaya/
├── backend/           API Express (server.js, db.js)
├── frontend/         Application React (src/App.js, Home.js, components/, contexts/)
├── Database/         4 fichiers de schéma / export SQL différents (incohérents, voir §9)
├── finish-import.js, fix-database.js, import-to-railway.js  scripts de migration ponctuels
├── (racine)
├── backend/init-db.js, backend/temp-init-endpoint.js      scripts de dev ponctuels
├── README.md, DEPLOYMENT.md, CONFIGURATION.md, PROJECT_SUMMARY.md  documentation
├── vercel.json (racine) + frontend/vercel.json           deux configurations Vercel
└── concurrentes
```

4. Architecture frontend

- `frontend/src/App.js` définit tout le routage. Presque toutes les routes destinées à l'utilisateur (`/` , `/categories/:id` , `/locations` , `/profile` , `/cart` , `/paidpages` , toutes les `/admin/*`) sont enveloppées dans `ProtectedRoute` , donc un visiteur non authentifié est immédiatement redirigé vers `/login` . Il n'y a aucune navigation publique du catalogue, contrairement à ce que décrit la documentation.
- `AdminRoute` ajoute une seconde vérification (`isAdmin`) par-dessus `ProtectedRoute` pour les six écrans CRUD d'administration (création / édition des catégories, destinations, offres).

- `frontend/src/contexts/AuthContext.js` est la source unique de vérité pour l'état de session et de panier :
 - Vérification de session : `GET /api/profile` avec `credentials: 'include'` (basé sur cookie, aucun jeton en JavaScript).
 - Panier : récupéré via `GET /api/cart` et poussé vers `PUT /api/cart` à chaque ajout / augmentation / diminution / suppression, le tout authentifié par cookie.
 - `login()` n'appelle pas lui-même `/api/login` — il suppose que `AuthForm` l'a déjà fait, et se contente de recharger le profil pour rafraîchir l'état.
- Le paiement (`Cart.js`) construit un lien direct WhatsApp (`wa.me/<numéro>?text=...`) à partir du contenu du panier — aucune commande réelle n'est enregistrée côté serveur au moment du paiement.
- `PaidPages.js` est une **fausse page de paiement** : elle recueille des informations de carte / PayPal (jamais validées ni envoyées nulle part), attend 2 secondes via `setTimeout`, puis affiche un message de succès et vide le panier. Rien n'est réellement débité ni enregistré.

5. Architecture backend

(`backend/server.js` , 830 lignes)

- CORS restreint à `FRONTEND_URL` avec les identifiants (credentials) activés ; parsing JSON ; parsing des cookies.
- Authentification : bcrypt (10 rounds) pour le hachage des mots de passe, JWT signé avec `JWT_SECRET` (expiration 1 h), stocké dans un cookie `httpOnly`, `sameSite=Strict` (l'attribut `secure` dépend de `NODE_ENV`).
- Limitation de débit : 5 requêtes / 15 min sur l'inscription, la connexion et le renvoi de vérification.
- La vérification de l'e-mail est obligatoire avant la connexion (indicateur `isVerified` + `verificationToken` à usage unique).
- Groupes de routes : authentification (`/api/register`, `/api/login`, `/api/logout`, `/api/verify-email`, `/api/resend-verification`, `/api/profile`), CRUD admin (`/api/admin/users` [GET], `/api/admin/categories` [POST/PUT/DELETE], `/api/admin/offers` [POST/PUT], `/api/admin/locations` [POST/PUT/DELETE]), catalogue (`/api/categories`, `/api/categories/:id`, `/api/categories/:id/offers`, `/api/locations`, `/api/locations/:slug`, `/api/locations/:slug/offers`, `/api/offers/:id`), panier (`GET / PUT /api/cart`) et contact (`POST /api/contact`).
- Toutes les écritures admin passent par `authenticateToken` + `authorizeRole(['admin'])`, correctement appliqué côté serveur (pas seulement masqué dans l'interface).
- Les requêtes SQL utilisent des paramètres préparés (`?`) partout — aucune surface d'injection SQL évidente.

6. Modèle de base de données

Vérifié à partir de `Database/import_railway.sql`, le schéma qui correspond réellement aux requêtes exécutées.

Table	Colonnes clés
-------	---------------

users	id, username, email, password (hash bcrypt), role (customer / admin), isVerified, verificationToken
categories	id, name, slug, icon, description
locations	id, name, slug, region, image
offers	id, title, description, price, infos_price, image, duration, service_type_id → categories, location_id → locations
cartitems	id, user_id → users, offer_id → offers, quantity (unique par utilisateur + offre)

Les clés étrangères utilisent **ON DELETE CASCADE** de bout en bout.

7. Parcours utilisateur et administrateur

1. Le visiteur s'inscrit → le backend envoie un e-mail avec un lien de vérification.
2. Le clic sur le lien met **isVerified=TRUE** et redirige vers **/verification-success**.
3. La connexion positionne le cookie JWT httpOnly ; le frontend recharge **/api/profile** pour renseigner **user**.
4. L'utilisateur parcourt les catégories / destinations, ajoute des offres au panier (synchronisé vers **cartitems** à chaque modification).
5. Le paiement ouvre WhatsApp avec la commande pré-remplie — aucun paiement réel ne s'effectue dans l'application (voir §4).
6. L'administrateur (role= **admin**) dispose en plus des écrans CRUD pour les catégories, destinations et offres, protégés par les points d'accès **/api/admin/***.

8. Observations intéressantes

Ce que l'historique git et le style du code révèlent au-delà du simple inventaire technique.

- **Un chantier de déploiement concentré sur deux journées.** Sur les 32 commits de l'historique, 21 (les deux tiers) sont datés du 1^{er} janvier et du 27 janvier 2026, et concernent presque exclusivement des ajustements de déploiement Railway / Vercel plutôt que des fonctionnalités : 4 commits sont même de simples « *Trigger Vercel rebuild* » sans aucun changement de code. Le développement fonctionnel proprement dit (inscription, panier, CRUD admin) s'est au contraire étalé bien plus lentement entre septembre et novembre 2025. Cette chronologie explique directement pourquoi les scripts et points d'accès temporaires identifiés en §9 n'ont jamais été nettoyés : ils ont tous été ajoutés pendant ce sprint de mise en production sous pression, où corriger un problème de schéma sur Railway avait la priorité sur la propreté du dépôt.
- **Deux parcours de paiement concurrents, sans arbitrage visible.** L'application propose à la fois un vrai bouton « Envoyer notre commande par WhatsApp » (qui fonctionne réellement) et une page **/paidpages** de paiement par carte entièrement simulée (qui ne débite jamais rien). Rien dans le code ni dans la navigation ne tranche lequel des deux est censé être le parcours d'achat officiel — un visiteur pourrait légitimement s'y perdre.
- **Un mélange assumé français / anglais dans le code.** Les commentaires, messages d'erreur API et textes d'interface sont presque entièrement en français (cohérent avec un marché marocain

francophone), tandis que les noms de variables, fonctions et routes restent en anglais. C'est un choix cohérent et courant dans ce contexte, mais cela signifie que toute contribution future devra respecter ce mélange plutôt que de le « corriger » vers un seul idiome.

- **Le choix de WhatsApp plutôt qu'un vrai prestataire de paiement est probablement un choix métier assumé, pas un oubli.** Pour une agence vendant des expériences touristiques sur devis (guides, excursions, hébergement), une confirmation humaine par message est souvent plus adaptée qu'un paiement immédiat par carte. Le risque n'est donc pas le choix lui-même, mais le fait que la page `/paidpages` laisse croire à tort qu'un système de paiement réel est en place.
- **Un commit de fusion sur un dépôt a priori solo.** Le commit « *Merge branch 'main' of ...* » suggère que le projet a été travaillé depuis au moins deux machines différentes, en poussant directement sur `main` sans branche de travail intermédiaire — ce qui explique en partie l'enchevêtrement de correctifs de déploiement observé le 27 janvier, plusieurs environnements locaux ayant apparemment tenté de corriger le même problème en parallèle.

9. Problèmes confirmés, classés par sévérité

CRITIQUE — Secrets versionnés dans git

Le fichier `backend/.env` est suivi par git (confirmé via `git ls-files`), bien qu'il figure dans `.gitignore` — il a probablement été commité avant l'application de la règle d'exclusion, ou ajouté avec `-f`. Il contient `DB_PASSWORD`, `JWT_SECRET`, `EMAIL_PASS`, `RECIPIENT_EMAIL`, etc. De plus, trois scripts situés à la racine — `finish-import.js`, `fix-database.js`, `import-to-railway.js` — sont eux aussi suivis par git et contiennent une `DATABASE_URL` **Railway de production, en clair** (nom d'utilisateur + mot de passe + hôte complets) en valeur de repli par défaut.

Action à mener : faire tourner immédiatement le secret JWT, le mot de passe de la base de données et le mot de passe d'application Gmail ; puis supprimer ces fichiers de l'historique git (les supprimer seulement à partir de maintenant ne suffit pas — `git log` exposerait toujours les anciennes valeurs), et retirer `backend/.env` du suivi (`git rm --cached`).

ÉLEVÉE — Point d'accès dupliqué et dangereux de réinitialisation de la base

`GET /api/init-database/:token` est défini **deux fois** dans `server.js` (autour des lignes 647 et 736). Les deux sont publics, protégés seulement par un jeton statique trivialement devinable, codé en dur dans le source lorsque `INIT_DB_TOKEN` n'est pas défini. Si l'un des deux est appelé avec ce jeton, il **supprime et recrée** `users`, `categories`, `locations`, `offers`, `cartitems` — détruisant toutes les données de production — et les reconstruit avec un schéma utilisant des noms de colonnes différents (`category_id`, `regular_price` / `member_price`, `image_url`, `is_verified`, `verification_token`) de ceux attendus par le reste de l'application (`service_type_id`, `price` / `infos_price`, `image`, `isVerified`, `verificationToken`). Déclencher cette route effacerait les données et casserait toutes les autres routes qui interrogent les anciens noms de colonnes.

Action à mener : supprimer entièrement les deux définitions de route dans `server.js` (code d'amorçage ponctuel qui n'a plus lieu d'être). Si une (ré)initialisation de la base est de nouveau nécessaire, la faire via un script local non accessible par HTTP.

ÉLEVÉE — La page de profil est silencieusement cassée

`frontend/src/components/Profile.js` extrait un `token` depuis `useAuth()` et envoie `Authorization: Bearer ${token}`. Or `AuthContext.js` n'expose plus de `token` (l'authentification repose désormais uniquement sur les cookies) — donc `token` vaut toujours `undefined`, la garde `if (!token)` se déclenche immédiatement, et la page de profil affiche systématiquement « Veuillez vous connecter », même pour un utilisateur connecté. C'est un résidu du code antérieur au passage à l'authentification par cookie `httpOnly`.

Action à mener : réécrire `Profile.js` pour appeler `GET /api/profile` avec `credentials: 'include'` comme le reste de l'application (ou simplement utiliser `user` directement depuis `useAuth()`, puisque le contexte contient déjà les données de profil).

MOYENNE — Des routes décrites comme publiques sont en réalité toutes protégées

La documentation (README/CONFIGURATION) décrit la navigation dans les catégories / offres / destinations comme une fonctionnalité publique de marketplace, mais `App.js` enveloppe `/`, `/categories/:id`, `/locations`, `/locations/:slug` dans `ProtectedRoute`, imposant une connexion avant de voir le moindre contenu. C'est un choix produit qui peut être légitime, mais il contredit actuellement la documentation écrite et nuit probablement à la conversion (un visiteur ne peut voir aucune offre avant de créer un compte).

MOYENNE — Fichiers de schéma de base de données incohérents

`Database/` contient quatre fichiers SQL différents (`almaya_local_export.sql` , `almaya_production.sql` , `structure_only.sql` , `import_railway.sql`) qui ne concordent pas tous entre eux, et aucun ne correspond au nom de fichier annoncé dans le README (`almaya_complete.sql` , qui n'existe pas dans le dépôt). Seul `import_railway.sql` correspond aux colonnes réellement utilisées par les requêtes actives de `server.js` .

Action à mener : supprimer les fichiers obsolètes ou incorrects et ne conserver qu'un schéma canonique unique, référencé de manière cohérente dans la documentation.

FAIBLE — Encombrement du dépôt

- Deux configurations Vercel (`/vercel.json` et `/frontend/vercel.json`) avec des stratégies de build différentes — source de confusion sur celle réellement prise en compte selon les paramètres du projet Vercel.
- `finish-import.js` , `fix-database.js` , `import-to-railway.js` , `backend/init-db.js` , `backend/temp-init-endpoint.js` sont des scripts ponctuels de migration / débogage laissés dans le dépôt (et, comme indiqué dans le point critique ci-dessus, ils exposent des identifiants). Ils n'apportent plus aucune valeur maintenant que les données ont été importées et devraient être supprimés.
- L'interface panier / paiement affiche les prix avec le suffixe € alors que l'activité (numéro WhatsApp `+212...` , destinations marocaines) est clairement libellée en MAD — probablement un espace réservé oublié plutôt qu'un choix intentionnel.
- Le champ `name` de `frontend/package.json` vaut `"frontend2"` , un reliquat d'un renommage.

10. Points forts

- Les mots de passe sont hachés avec bcrypt ; le JWT réside dans un cookie httpOnly et sameSite plutôt qu'en localStorage — aucun vecteur de vol de jeton par XSS.
- La vérification de l'e-mail est obligatoire avant la première connexion.
- Toutes les écritures admin sont contrôlées côté serveur par rôle, pas seulement masquées dans l'interface.
- Les requêtes SQL sont paramétrées — aucune surface d'injection détectée.
- L'état du panier est persisté côté serveur par utilisateur plutôt qu'uniquement dans le navigateur.
- Organisation en monorepo adaptée à l'intégration continue, avec frontend et backend déployables séparément et des configurations Railway / Vercel cohérentes.

11. Prochaines étapes recommandées, par ordre de priorité

1. **Faire tourner tous les secrets** actuellement présents dans le `backend/.env` commité et dans les trois scripts contenant la `DATABASE_URL` codée en dur (mot de passe DB, secret JWT, mot de passe d'application Gmail) — les considérer comme compromis puisqu'ils figurent dans l'historique git.

2. **Supprimer les points d'accès dupliqués** `/api/init-database/:token` de `server.js` — il s'agit d'un risque réel de perte de données présent dans le code de production.
3. **Corriger** `Profile.js` pour utiliser l'appel `/api/profile` basé sur cookie (ou lire `user` depuis le contexte) au lieu du chemin mort par `token` / en-tête Authorization.
4. **Supprimer les scripts ponctuels restants** (`finish-import.js` , `fix-database.js` , `import-to-railway.js` , `backend/init-db.js` , `backend/temp-init-endpoint.js`) et regrouper `Database/` en un seul schéma conforme à la production.
5. Décider clairement si la navigation doit être publique ou réservée aux utilisateurs connectés, et aligner la documentation et les routes sur ce choix.
6. Réconcilier les deux fichiers `vercel.json` ainsi que la devise affichée (€ vs MAD).